Activity Frames: Deterministic Screen-Activity Compilation for Agent Memory and Replay

Nossa Iyamu*

Abstract

Computer-use agents pay full frontier inference to rederive routines their user has already performed, because an agent's memory today records what the user said, not what the user did. We compile passively captured screen activity into agent memory with a deterministic, zero-model pipeline: it segments a local capture stream into typed *activity frames*, bounded episodes carrying application, site, timing, input volume, and evidence pointers back to the raw rows, with no model in the loop, so the output is byte-identical, cacheable, and mechanically auditable. On one professional's single-user corpus of 128,756 frames over 51 active days, the compiler reduces a day of raw capture to a prompt-ready context block 86× smaller in 68 ms, and an agent reading that block answers questions about the day at 98.4% accuracy (Wilson 95% CI 91.7–99.7%) against an independent oracle, versus 66–80% for an LLM summary of the same capture, a mid-tier model reading the block matching a frontier one.

The same compiler doubles as a demand-side cost instrument. Read off passive, pre-delegation human activity rather than agent rollouts, it supplies two parameters that agent-cost models assume but, to our knowledge, have not measured: the Routine Overhead Ratio R and the routine recurrence h. We report first values of R, a modeled upper bound, at 60–343×, and a delegable recurrence of 9.0% in-sample and 7.7% out-of-sample, for a realistic all-fleet token ceiling near 8%; a compiled routine replays deterministically with the model out of the loop, demonstrated live at zero model tokens on a guard-matched hit. Schema, compiler, and evaluation harness are open.

Keywords: LLM agents, episodic memory, agent cost, routine replay, computer-use agents, screen capture, Model Context Protocol

1 Introduction

An LLM agent asked "what should I prioritize today?" answers from what it can see: the conversation, some files, perhaps a calendar. It cannot see that its user spent the morning inside a pull request, switched contexts forty times after lunch, or abandoned a draft email at 16:40. Position work has argued that episodic memory, a record of experience situated in time, is the missing piece for long-term LLM agents [1], and surveys of agent externalization treat memory as a first-class architectural concern [2]. Yet in practice, agent memory today means conversation memory: what the user told the model, not what the user did [3–5].

The raw material for behavioral episodic memory already exists. Automated time trackers have recorded application focus for a decade [6]; operating systems now ship continuous capture, with Microsoft Recall storing periodic snapshots on Copilot+ PCs [7]; and OpenAI's Chronicle preview builds memories from recent screen content for its Codex assistant [8]. Capture, in other words, is commodity. Consumption is not. A day of event-driven capture on our corpus yields roughly two thousand snapshot rows, each one asserting only that at time t, application a displayed window w at URL u. Two consumption strategies dominate. Raw search hands the agent a flat result list and leaves sessionization, deduplication, and duration accounting to the model at inference time; we measure this cost at 126,812 tokens for a single day (Section 6). LLM summarization compresses well but imports the weaknesses of its summarizer: per-run cost, non-determinism, degraded reliability over long inputs [9], and the possibility of inventing activity that never happened. Neither yields memory an agent can cache, audit, or trust.

This paper proposes the missing middle: *deterministic compilation*. In the same way a compiler turns instructions into structured artifacts without opinion, we turn snapshot streams into *activity frames*: bounded episodes with an application and site, a start and end, dwell-based active time, typed page references, input volume, and evidence pointers back to the raw rows. No model participates. The same database and window always produce the same document, so episodic memory becomes free to rebuild, safe to cache, and mechanically auditable. Interpretation is not banned but quarantined: a second schema tier accepts inferred labels only when they are namespaced, confidence-tagged, and linked to the measured evidence that supports them.

Our contributions are:

*Independent Researcher. Correspondence to nossa.iyamu1@gmail.com.

- **A two-tier schema** for representing human computer activity to agents, separating measured fact from tagged inference, with coverage gaps and blind spots as mandatory document elements (Section 3).
- **Deterministic compilation rules** (dwell crediting, session gaps, flicker merging, nearest-frame attribution, coordinate-based click resolution, total URL-to-entity typing) that require no learned components (Section 4).
- **An open reference implementation**: a dependency-free Python compiler with a provisioned local capture engine, a CLI, and a Model Context Protocol (MCP) server exposing six tools to any MCP-capable agent [10, 11] (Section 5).
- **An empirical characterization** on a 61-day live corpus: token cost against raw rows (86× reduction for prompt-ready context), 68 ms full-day compilation, byte-identical reproducibility, entity-typing coverage, and a duration distribution that separates genuine short attention bouts from single-snapshot transits and multi-monitor effects (Section 6).
- **A downstream question-answering benchmark** against an independent oracle, run at two model tiers, showing that an agent answers more accurately from the compiled block than from raw rows or an LLM summary of the same capture at both tiers, that the block lets a mid-tier model match a frontier one, and that a stronger model narrows but does not close the gap; the harness is released for rerunning against any model (Section 6.2).
- **A demand-side cost instrument**: an open, deterministic measurement of the Routine Overhead Ratio R (modeled numerator, measured denominator) and desktop routine recurrence h, read from passively captured human activity rather than agent rollouts, with a parametric replay executor and a first live proof-of-concept, reported on one user's corpus (Sections 7–7.4).

2 Related Work

2.1 Memory Systems for LLM Agents

MemGPT virtualizes context OS-style, paging conversation history through a bounded window [3]; production layers consolidate salient facts from dialogue [4]; Zep organizes memory as a temporal knowledge graph [5]; E-mem reconstructs episodic context from execution traces [12]; and hierarchical procedural memory distills skills from agent trajectories [13], threads a recent review unifies as externalization [2]. All ingest what flowed *through the agent*, messages, tool calls, task traces, and memory-construction studies show that granularity and structure affect retrieval quality [14], supporting typed episodes over flat logs. Agent Workflow Memory induces routines too, but from agent rollouts rather than human activity [15].

2.2 Screen Capture as Agent Memory

A 2025–2026 wave targets our exact goal by the opposite method. MIRIX runs a multi-agent memory system over continuous screenshots with cloud model calls [16]; FOCAL invokes a local vision-language model to write session summaries from on-device capture, reporting a 60% token cut [17]; ProAgentBench segments 500+ hours of capture on application switches before model-based annotation [18]; SummAct summarizes interaction traces into intentions with an LLM [19]; and OmniQuery augments captured personal media for QA [20]. In every case a model sits inside the memory-construction loop, so the memory inherits per-run cost, non-determinism, and possible hallucinated episodes. Activity frames are the deterministic counterpoint: no model in the compile path, byte-identical output, and interpretation quarantined in a separate evidence-linked tier, the first deterministic desktop instantiation of the acquisition layer episodic-memory advocates call missing [1].

2.3 Desktop Activity, GUI Agents, and Memory Trust

Deriving task structure from interaction logs is a two-decade ambition: TaskTracer tied window, file, and clipboard events to declared tasks [21], and SWISH clustered windows into tasks from titles and switching [22]. Interruption science established the fragmentation our compiler measures, three-minute working spheres [23] and content switching about every 19 s with 75% of segments under a minute [24]; these are behavioral rates from their instruments, and our comparable figure is the post-transit median frame of 0.9 min (Section 6), not our 6.1 s median inter-capture gap, which is recorder cadence rather than a human switching rate. Methodologically our pipeline descends from lifelogging's threshold segmentation [25], whose critique that unstructured total capture serves nobody [26] is the failure mode we exist to prevent, plus web sessionization's inactivity timeouts [27], process-mining event abstraction [28], robotic process mining's UI-log routine identification [29], and the data-to-text faithfulness tradition [30]; but those pipelines end in process models or prose for humans, ours in typed memory for an agent. Separately, a large body gives agents screens as an *action* surface, GUI-agent surveys cataloging accessibility-tree, OCR, and vision perception [31, 32], and generative agents showing a remembered observation stream enables long-horizon behavior [33]; we compile the inverse, what the human did, an acquisition problem personal-agent surveys call open [34]. Deployed capture leaves the same gap: ActivityWatch is content-blind [6], Recall exposes snap-

shots to the user but no agent API [7], Chronicle feeds one walled assistant [8]. Finally, agent memory is now an attack surface, memories poisoned to steer behavior [35] or probed by membership inference [36], motivating two properties we make structural: evidence pointers back to raw rows, and a hard measured/inferred boundary, so a poisoned or hallucinated label cannot pass as fact. Context-engineering and long-context degradation results both argue for compact structured context over raw dumps [9, 37].

2.4 Agent Skills, Trajectories, and Cost

Three adjacent lines each need one of our parameters but source it differently. *Skill induction* turns an agent's own successful trajectories into reusable skills: Agent Workflow Memory from web rollouts [15], SkillWeaver from self-execution [38], Agent Skill Induction as programs [39], and PreAct as continual self-improvement [40]. All read the *supply* side, what an agent did while acting, and so observe a routine only after an agent has already performed it at full cost, and only for tasks it can complete. *Trajectory-acquisition* channels manufacture training data and report a price: Explorer at ≈$0.28 each [41], AgentTrek at ≈$0.55 [42], and Watch & Learn [43] and cotomi Act [44] at low marginal cost; but a synthesized trajectory exists only where the generator can drive the task to success, so authenticated, private-state routines are systematically under-covered, and Agent Data Protocol's channel taxonomy [45] omits both passive capture and a price column. *Cost frameworks* price the rest: FrugalGPT for model cascades [46], Cost-of-Pass for expected cost per success [47], AI Agents That Matter for cost as a first-class axis [48], and the Holistic Agent Leaderboard for real rollout economics (≈$1.84/task) [49]. Together they supply an amortized per-task cost of a memory-backed agent, which we adopt as a cited frame, not a contribution:

$$\begin{aligned}\mathbb{E}[\$/\text{task}] = (1-hq)\,\tfrac{C_{\text{miss}}}{p} + hq\,(C_{\text{hit}}+C_{\text{verify}}) \\ + h(1-q)\,C_{\text{wrong}} + \tfrac{C_{\text{write}}}{N},\end{aligned} \tag{1}$$

where h is recurrence, q the fraction of hits correctly matched, p the base success rate, N the reuse count, and the C-terms per-branch costs. Every term is already priced in the works above; the one input none of them measures is h on the *pre-delegation* passive corpus. Rollout-priced leaderboards observe the hit rate of tasks an agent was given and completed, a survivorship-biased quantity; the demand-side h, how often the user's real work repeats whether or not an agent could do it, is what our instrument supplies, and R replaces the assumed $C_{\text{miss}}/C_{\text{hit}}$ ratio with a measured one. Activity frames are thus the pre-delegation, demand-side instrument that skill induction, acquisition channels, and cost leaderboards each presuppose but none provides.

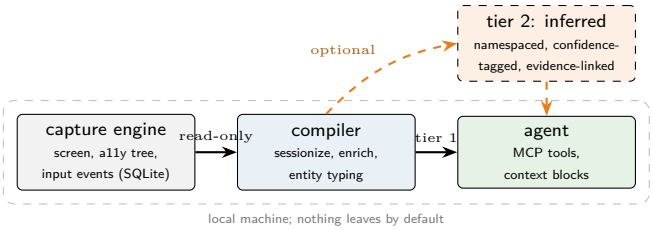

Figure 1: The two-tier architecture. The measured tier (blue) is produced entirely by deterministic code reading the capture database. Interpretation (orange) is an optional extension that must be namespaced, confidence-tagged, and evidence-linked; stripping it always leaves a valid measured document.

3 The Activity Frames Schema

3.1 Design Principles

Four principles fix the schema's character. **Measured, not guessed**: every field in a standard document is derivable by deterministic code from capture data; there are no intent labels, because code cannot observe intent. **Reproducible**: identical inputs must yield identical documents, making memory cacheable and diffable. **Evidenced**: every frame carries pointers to the raw rows it was compiled from. **Honest about absence**: periods without capture are reported as gaps, and every document carries a `blind_spots` list stating what the pipeline systematically cannot see. Consumers must treat uncovered time as unknown, never as inactivity.

3.2 Document Structure

A document describes one query window and contains four parts: a `coverage` section (first and last activity, active minutes, span, capture gaps over five minutes), a chronological list of `frames`, a `blind_spots` list, and provenance metadata (`schema_version`, generation time, source recorder). A frame is one bounded stretch of attention in a single context, keyed by the pair (application, site), where site is the URL host for browser activity and absent otherwise. Figure 2 shows a representative frame.

3.3 Typed Page References

Browser frames carry `pages`: typed references produced by deterministic URL parsing. A reference has a `kind` (what sort of page), an optional `entity` (the human-relevant identifier), and a view count. Standard kinds include `profile`, `company`, `people_search`, `search`, `repo`, `pull_request`, `issue`, `doc`, `email`, `video`, `post`, `ai_chat`, and `local_dev`. The mapping is total: a URL matched by no site parser falls back to a generic `page` reference with its domain, so typing never loses data.

```
- id:  f-0007
  app:  "Google Chrome"
  site:  "linkedin.com"
  start:  "20:24:04"  end:  "20:42:11"
  duration_min:  18.0   wall_min:  21.5
  pages:
    - {kind:  people_search, entity:  "cto paris",
count:  2}
    - {kind:  profile, entity:  "jane-doe"}
    - {kind:  company, entity:  "acme"}
  input:  {keys:  214, clicks:  31}
  interruptions:  [{app:  "Slack", seconds:  12}]
  evidence:  {frame_ids:  "99871..100147"}
```

Figure 2: A single activity frame (YAML, entities anonymized). Every field is computed by code; the evidence pointer names the raw snapshot rows the frame was compiled from.

Kinds are open for extension but must remain deterministic functions of the URL.

3.4 Tier 2: The Inferred Extension

Tools may add interpretation, such as task labels or project clusters, under three schema-enforced rules: inferred content lives in a namespaced `inferred` block, carries a `confidence` tag drawn from {`high`, `medium`, `speculative`}, and names its `evidence`: the measured fields or raw rows supporting it. A consumer can always strip the `inferred` block and be left with a purely measured document. The reference implementation emits tier 1 only; we consider the boundary itself, not any particular inference method, to be the contribution.

3.5 Privacy Rule

Input *volume* (keystroke, click, and copy counts) is part of the standard document. Input *content* (typed text) must be excluded by default and included only on an explicit operator opt-in. This is a schema requirement, not an implementation courtesy: a conforming producer cannot silently emit content.

4 Deterministic Compilation

4.1 Setting

The capture engine is event-driven with a heartbeat: it stores a snapshot row on interaction and on screen change (clicks, application switches, visual changes), plus a periodic row after roughly 30 seconds without input. Heartbeat rows are 19% of our corpus; they matter because they keep the stream alive while the user reads, watches, or steps away from an awake display, and every downstream time measure inherits that. Each monitor records its own stream; within a stream the median inter-frame gap is 6.1 s and the 90th percentile is 30.7 s (Section 6). A row carries a timestamp, application, window title, and URL when the focused application is a browser. Input events (keystrokes, clicks, clipboard, application switches) arrive in a parallel stream. Compilation consumes these streams for a query window, segmenting each monitor independently, and emits the document of Section 3.

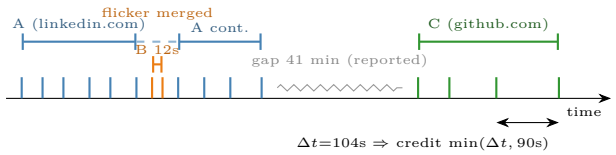

Figure 3: Segmentation on one timeline. Ticks are snapshot rows. A 12-second detour (B) folds into the surrounding frame as a recorded interruption; a long silence becomes a reported coverage gap; sparse snapshots earn at most the 90 s dwell cap.

4.2 Sessionization

Three constants govern segmentation, chosen from the capture cadence rather than tuned on outcomes. **Dwell**: a frame contributes $\min(\Delta t, 90\,\text{s})$ of active time, where Δt is the gap to the next frame in its monitor's stream. Because the engine emits a ∼30 s heartbeat during input-free stretches, dwell measures *screen tenure*: how long a context stayed frontmost on an awake display. That includes reading and watching, which produce no input, but also stretches where the user has stepped away while the display stays on; Section 8 quantifies this. The cap, roughly three heartbeat periods, bounds the credit a single frame can earn when capture stalls or the display sleeps; it does not, and cannot, distinguish attention from presence. Consumers who need interaction-gated time can compare each frame's reported input volume against its duration. Segmentation runs per monitor, so a context visible on two monitors at once earns tenure on both; the schema discloses this as a blind spot and Section 6.2 shows the consequence. **Session gap**: a gap above 300 s closes the current frame and becomes a candidate coverage gap; no dwell is credited across it. **Flicker merge**: the pattern $A \to B \to A$, where B lasts at most 20 s of wall time and no session gap intervenes, collapses into a single A frame. Crucially, B is not discarded: it is recorded on the merged frame as an `interruption` with its measured seconds, and its time is *not* added to A's active duration. Nothing is hidden and nothing is double-counted. Algorithm 1 summarizes the procedure; Figure 3 illustrates all three rules on one timeline.

4.3 Enrichment

Raw input events have three reliability defects that code can repair. First, **stale attribution**: the recorder some-

Algorithm 1 Frame segmentation (one pass, then flicker merge)

Require: snapshots $f_1 \dots f_n$ of one monitor's stream, sorted by time; constants D=90, G=300, F=20 (seconds)

$S \leftarrow []$; $cur \leftarrow \perp$
for $i = 1$ **to** n **do**
  $k_i \leftarrow (\text{app}(f_i), \text{site}(f_i))$; $\Delta \leftarrow t(f_{i+1}) - t(f_i)$
  if $cur = \perp$ **or** $k_i \neq \text{key}(cur)$ **then**
    append new segment cur with key k_i to S
  end if
  extend cur with f_i
  if $\Delta \leq G$ **then**
    $\text{active}(cur) \mathrel{+}= \min(\Delta, D)$
  else
    $cur \leftarrow \perp$ {session break; candidate gap}
  end if
end for
for consecutive A, B, A' in S with $\text{key}(A)$=$\text{key}(A')$ **do**
  if $\text{wall}(B) \leq F$ **and** no session break around B **then**
    merge A' into A; record B as interruption of A
  end if
end for
return S

times tags an event with the previously focused application. Each event is therefore re-attributed to the temporally nearest snapshot (binary search over the frame stream), whose application, window, and URL are authoritative; the attribution distance is kept in milliseconds so downstream consumers can judge it. Second, **anonymous clicks**: many click events carry coordinates but no element name. We resolve them against the snapshot's recorded element tree: exact containment first (smallest containing element wins), then a ± 40 px tolerance ring, then a coarse screen-zone fallback; every resolution is tagged `exact`, `tolerance`, or `zone`, and unresolvable clicks stay unresolved rather than being guessed. Third, **keyboard-layout mismatch**: some capture stacks record physical key positions decoded as QWERTY while the user types another layout. An explicit, operator-supplied translation map repairs this; it is identity by default and never inferred.

4.4 Entity Typing

Site parsers map URLs to the typed references of Section 3: path and query parsing only, no fetching, no models. Resolution proceeds in layers: a bespoke parser for the host (the reference implementation ships more than twenty, covering professional networks, code hosting, search, documents, mail, maps, video, social, events, dashboards, AI chat, and local development), then a generic search-parameter detector, then a subdomain-and-path heuristic that types common infrastructure pages (sign-in, dashboard, email, calendar, meeting) even for sites without a bespoke parser, and finally a total fallback that guarantees every URL maps to something. Because every layer is a pure function of the URL, adding coverage is a contribution reviewable line by line.

Table 1: MCP tool surface of the reference implementation.

Tool	Returns
`get_context`	Compact chronological context block for the last N hours, sized for inclusion in a system prompt
`get_activity`	Full schema-v1 document (JSON) for a day or window
`get_day_summary`	Coverage plus per-app ledger (minutes, sessions, longest session)
`get_patterns`	Repetitive workflows over the last N days
`get_communications`	Email/messaging surfaces with the window titles seen on each (measured tier; titles only)
`get_steps`	Ordered click-by-click script behind one activity frame, for replay

5 Reference Implementation

The reference implementation [10] is a Python package (MIT license) with three parts. The **capture engine** is provisioned on demand by `aframes record`: a pinned, MIT-licensed, open-source build that records application focus, window titles, URLs, the accessibility tree, and input events into a local SQLite database, entirely on-device, with audio capture off by default. Operators already running a compatible recorder can skip it and point the compiler at any existing database. The **compiler** has zero runtime dependencies, opens the database read-only, and exposes the document builders plus emitters for JSON, YAML, Markdown, and a compact plain-text context block designed for system prompts. The **MCP server** [11] is a hand-rolled stdio JSON-RPC implementation, also dependency-free, exposing the six tools of Table 1 to any MCP client. A workflow-pattern detector (repeated clicks, URL loops, action sequences, application-switching habits, daily habits) rounds out the surface.

6 Empirical Characterization

We characterize the system on the author's own live corpus: 61 calendar days of capture (46 days with activity) comprising 109,735 snapshot rows, 214,360 input events, and 8.4M element-tree rows across 54 applications (Table 2). The corpus is frozen: the recorder was migrated to a new database after the last captured day, so every number below is reproducible against an immutable file. The paper reads from this corpus at two freezes, labeled where used: this systems half uses the 2026-07-10 freeze (61 calendar / 46 active days, 109,735 frames), while the

Table 2: Live capture corpus used throughout Section 6.

Calendar span	61 days
Days with activity	46
Snapshot rows (frames)	109,735
with URL	59,458
idle-heartbeat rows	20,429 (19%)
Input events	214,360
Element-tree rows	8,395,885
Median intra-monitor gap	6.1 s
90th-percentile gap	30.7 s
Distinct applications	54

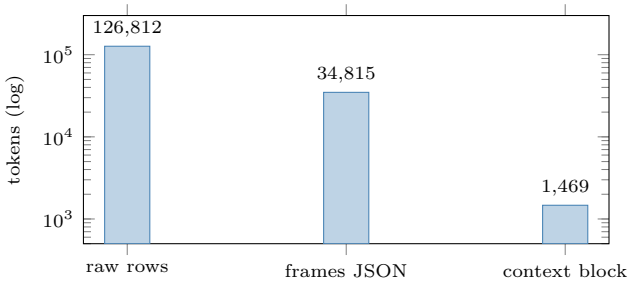

Figure 4: Token cost of one full day under three representations (cl100k_base). Deterministic compilation yields an 86× reduction for prompt-ready context, at zero inference cost.

overhead measurements of Section 7 use a later 2026-07-22 freeze (51 active days, 128,756 frames). This is a single-user corpus; we report it as a characterization of the mechanism, not a user study (Section 8).

6.1 Token Cost

For one representative full day (2,066 snapshot rows), we compare three representations an agent could receive, tokenized with the `cl100k_base` encoding. The raw rows, serialized as the JSON a search API would return, cost 126,812 tokens. The compiled schema-v1 document (222 frames at a 0.5-minute floor) costs 34,815 tokens, a 3.6× reduction that also relieves the agent of segmentation work. The compact context block costs 1,469 tokens, an 86× reduction, small enough to include in every system prompt (Figure 4). Both compiled forms are produced without any model call, so the reduction is free, and long-context results suggest the smaller representation is not merely cheaper but better used by the model [9].

6.2 Downstream Question Answering

Token cost is a means; the end is whether an agent can *answer questions about the user's day*. We test this directly. We evaluate on eight days chosen by a fixed rule: the seven most recent consecutive active days plus the nearest preceding day whose raw serialization overflows the model's context window (included to exercise that regime). For each day, ground-truth answers are computed by an *independent* SQL oracle over the raw tables: a documented inactivity-timeout dwell (credit each frame the gap to the next, capped at 60 s), deliberately not the activity-frames compiler, so the reference is not circular. Oracle and compiler measure the same construct, screen tenure: the capture heartbeat (Section 4) keeps input-free stretches credited, so "active minutes" throughout this section means time a context was frontmost on an awake display, not interaction time. Every representation is graded against that one construct, which keeps the comparison fair, but absolute magnitudes should be read as tenure. We validate the compiler against the oracle before using the oracle to grade anyone. On the seven benchmark days whose capture was complete when the oracle was frozen, the compiler's covered active minutes agree with the oracle's total dwell to a median of 0.9 minutes (all within 2.3); the eighth day's capture continued after the freeze, so it is compared only at its snapshot. Distinct-application counts agree exactly or within one on every day. Two caveats keep this agreement honest. First, it compares aggregates that both approximate covered wall time, so it validates coverage rather than per-application arithmetic. Second, the two systems intentionally differ on multi-monitor days: the compiler credits each monitor's stream (Section 4), while the oracle interleaves all monitors into one. On the context-overflow day, which has 133 dual-monitor minutes, the dominant application earns 435.8 minutes by the compiler's ledger but 346.1 by the oracle, a 26% divergence that the grading tolerance below happens to contain; switching the oracle's cap from 60 to 90 s moves its day total by under 6 minutes, so the divergence is the monitor convention, not the dwell cap. We report this rather than average over it. The generator emits 64 questions across five categories: which application dominated, how many active minutes in it, how many distinct applications, pairwise time ranking, whether a given domain was visited, and starting time, plus 16 *absent-fact probes* ("did the user open Photoshop / visit netflix.com?") that a faithful system answers negatively. We deliberately exclude two further question types. Longest-session length depends on the session-gap convention rather than a fact, and per-profile recall exceeds the compact block's token budget on busy days; each would penalize the compact block for a convention or a compression choice rather than for faithfulness, so we record the exclusion here to keep it on the books.

Two agents at different capability tiers, Claude Sonnet 4.5 and the larger Opus 4.5, each given no tools and answering only from the supplied text, answer every question from each of three representations: the raw rows, an LLM-generated summary of those rows written by that same agent, and the activity-frames output (the deterministic per-application ledger plus the compact context block, together ≈2k tokens). We note plainly that this hands the activity-frames agent measured per-application durations the other two must derive for themselves: that is exactly what the compiler is

Table 3: Downstream QA across two model tiers (8 days, 64 ground-truth questions, graded against the independent SQL oracle). "Acc." is overall accuracy; "Dur. err." is the mean absolute error on the dominant application's active minutes. The compiled block scores identically at both tiers, so a mid-tier model reads it as well as a frontier one; the raw-row and summary baselines improve with model strength but never catch it. Only the block is deterministic across runs and fits the context window on every day; on the busiest day the raw rows are 257k tokens, so both baselines are infeasible and report on 7 of 8 days (the block, on all 8).

	Sonnet 4.5		Opus 4.5	
Representation	Acc.	Dur. err.	Acc.	Dur. err.
Raw rows	82.1%	39.7%	91.1%	25.7%
LLM summary	66.1%	135.7%	80.4%	25.2%
Activity frames	**98.4%**	**7.3%**	**98.4%**	**7.3%**

for, but it means the quantitative questions test whether the deterministic arithmetic was already done, not only whether the agent can read. Answers are graded against the oracle with fixed tolerances (numeric within 30%; times within 45 minutes), and the ranking does not depend on them: at the Sonnet tier, a strict 10%/15-minute band leaves the three representations at 95.3%, 80.4%, and 66.1%, and the summary is so far off that widening the band to 50% lifts it only to 67.9%. Table 3 reports both tiers at the default tolerance.

Four findings. First, **the gap is quantitative, not categorical.** At both tiers all three representations score perfectly on the absent-fact probes: the models do not invent applications or websites (hallucination rate 0% throughout). The summary fails instead on *magnitudes*. At the Sonnet tier it answers time questions at 7.1% accuracy and mis-states the dominant application's active minutes by 135.7% on average, against 7.3% for activity frames (that residual is dwell-method rounding, not error). Fluent prose hides the damage: on 2026-07-05 the measured record (the oracle, with the compiler's ledger agreeing) is Google Chrome first at 161.6 dwell minutes and Cursor second at 143.9, over a day that ran 11:40 to 22:26, yet a Sonnet summary of that day names Cursor the primary application at "∼7 hours," inflating its 144 measured minutes by 2.9× and fabricating a nocturnal "6:40 PM–5:26 AM" session that spills into the next day. The activity-frames agent answered 160 minutes. Every day, summary, and graded answer is in the released harness output.

Second, **a stronger model narrows the gap but does not close it, and the compiled block erases the difference between the two.** The frontier Opus model reconstructs durations far better from prose: its summary reaches 80.4% overall and cuts the duration error to 25.2%. But that is still 3.5× the block's 7.3%, and reading the block the two models are indistinguishable (98.4% each), whereas reading raw rows or a summary the frontier model runs 9 to 14 points ahead of the mid-tier one. The block lets the cheaper model answer as well as the expensive one; the baselines make capability matter. The benefit of deterministic compilation is therefore largest exactly where compute is cheapest to deploy.

Third, **the summary is non-deterministic**: regenerating it three times per day produced three distinct texts on every day at both tiers, whereas the activity-frames block was byte-identical across three regenerations, so the summary's errors are not even stable enough to correct for. Fourth, **the baselines do not always run.** The busiest day serializes to 257k raw tokens, exceeding the context window, so both raw-row and summary consumption are infeasible; the ≈2k block is unaffected, which is why it alone reports on all eight days. Activity frames are not flawless: their one miss (98.4%, not 100, at both tiers) is a domain-recall question on the busiest evaluated day, where the compact block's budget drops a visited domain; that is the expected price of a bounded representation, and a recall question rather than a magnitude one. These accuracies are a single answering pass per representation per tier, at provider default sampling (model snapshots `claude-sonnet-4-5` and `claude-opus-4-5`, temperature and seed unpinned, so the non-deterministic baselines vary across re-runs); we release the full harness (oracle, question generator, and grader) so they can be repeated, with confidence intervals and further models, on any corpus.

6.3 Latency and Reproducibility

Compiling the same full day end to end (segmentation, enrichment-backed input accounting, entity typing, emission) takes a median of 68 ms over five runs on a consumer laptop (Apple Silicon), with a 65 to 72 ms range. Episodic memory at this cost can simply be rebuilt on every query. Reproducibility holds by construction and we verify it empirically: two independent compilations of the same window produce byte-identical documents once the generation timestamp is excluded. Determinism is what makes the memory cacheable, diffable across code versions, and testable in continuous integration.

6.4 Entity Typing Coverage

Across all 5,120 distinct URLs in the corpus, the layered parsers produce a non-generic typed reference for 81.3%, spanning 46 kinds; the remaining 18.7% fall back to the generic page reference with domain. The most frequent non-generic kinds are `profile` (1,106 distinct URLs), `search` (397), `email` (278), `event` (209), and `messaging` (200), reflecting that typed coverage concentrates precisely on the high-signal pages an agent most benefits from resolving. Coverage grows one pure function per site, so the long tail is closed incrementally by

contribution rather than by any learned component.

6.5 How Fragmented Is a Day?

Compiling the 43 days that produce compiled frames (of the 46 with any capture in Table 2; the other three carry only idle-heartbeat rows) yields 17,514 frames (median 361 per day) with a median active duration of 0.5 minutes: 68% of frames last under one minute (Figure 5). A number this stark demands decomposition before interpretation, because two mechanical effects sit inside it. First, 52% of the sub-minute frames (6,177) contain a single snapshot, with a median credit of 6.1 seconds: these are *transits*, the application an operator passes through on the way somewhere else, faithfully recorded but not attention episodes. Second, a further 20% (2,361) lie inside dual-monitor stretches, where each monitor earns its own stream by construction; 27% of captured minutes on this corpus have two monitors active. Restricting to frames with at least two snapshots, the median rises to 0.9 minutes and 52% remain under one minute; that residue is genuine switching. Read this way, the distribution is consistent with what interruption science has measured with dedicated instrumentation: three-minute working spheres [23] and 75% of laptop content segments under one minute [24]. We deliberately do not present the raw histogram as an independent replication: an earlier draft of this analysis segmented all monitors as one interleaved stream, which shreds dual-monitor stretches and pushed the median to 0.3 minutes, and strict segmentation always overstates fragmentation until transits are separated out. The lesson is the schema's, not just ours: consumers get a `min_minutes` floor (frames below it are omitted with the omission disclosed, never silently merged), the flicker-merge rule keeps sub-20-second detours from shredding genuine focus blocks while still recording them, and single-snapshot transits remain visible in the document precisely so that no compiler-side heuristic has to decide what counts as attention.

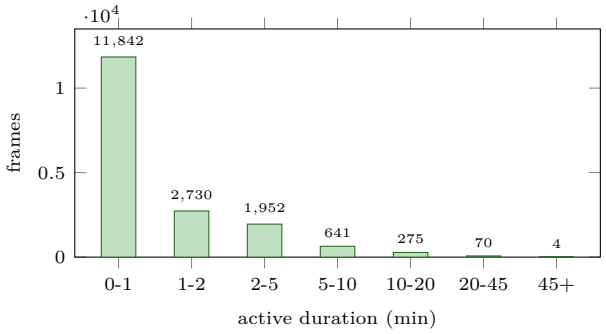

Figure 5: Active-duration distribution of all 17,514 frames across 43 days (per-monitor segmentation). Median 0.5 min, but 52% of the sub-minute bar is single-snapshot transits rather than attention episodes; excluding them the median is 0.9 min (see text).

7 The Routine Overhead Ratio

The compiler of Sections 3–5 was built to produce episodic memory, but it is also an *instrument*. Because it reduces a recurring stretch of capture to a short, replayable script by deterministic code alone, it lets us put a measured number on a quantity that agent-cost models assume but none of them observes: how much a computer-use agent overpays to re-derive a routine it has already done. We call that number the Routine Overhead Ratio R, and we report it, together with the recurrence h (Section 7.4), as the first readings of the two parameters those cost models are written in. Neither R nor h is offered as a principle; each is a measurement, taken with an open tool on one user's real work. The figures in this half are read from the same single-user author corpus characterized in Section 6, at a later freeze: Section 6's systems numbers use the 2026-07-10 freeze (61 days, 46 active, 109,735 frames), while the overhead and reproducibility numbers here use the freeze extended through 2026-07-22 (51 active days, 128,756 frames, 232,898 input events). Between the two freezes the accessibility-tree coverage rose from about 41% to 81.5% of frames, which strengthens the structured grounding available to replay.

7.1 Definition

A *routine* is a frequent action n-gram: a contiguous sequence of k UI actions ($3 \le k \le 60$) that recurs at least three times within the corpus, restricted to sequences that name at least two distinct targets. Sessions are cut at inter-action gaps over 90 s. The two-named-target restriction is declared in advance, not tuned after the fact: without it, degenerate single-symbol repeats (a held key, a scroll loop) would satisfy "recurs" trivially and inflate every ratio, the granularity trap that any repetition metric on raw input invites. It is the same specificity that separates the delegable rate h from the raw rate in Section 7.4.

For a routine of k steps we compare two token costs. The numerator is **modeled**. A memoryless, screenshot-driven agent re-derives the routine one step at a time, and per step consumes one screenshot, a fixed context read, and a fixed reasoning write:

$$C_{\text{agent}}(k) \;=\; k\left(\frac{w\,h}{750} \;+\; 350 \;+\; 180\right), \qquad (2)$$

the three per-step terms being the screenshot priced by Anthropic's $wh/750$ image-token rule for a $w \times h$ capture, a 350-token context read, and 180 tokens of reasoning output. At a typical 1512×982 capture this is about 2,500 tokens per step (1,979 image + 350 + 180), dominated by the screenshot. We are explicit that C_{agent} is what published screenshot-driven agent loops *would*

spend, computed without executing an agent, and that it is an *upper bound*: it assumes a screenshot baseline with no cross-step prompt caching, so an agent that reuses context across steps or grounds on the accessibility tree instead of a fresh screenshot spends fewer tokens per step and R narrows accordingly. Section 8.4 reports a live in-loop comparison against an accessibility-tree agent that saved only ≈14%, and reserves the full live three-arm billing. The denominator is **measured**, and we report it as a *ladder* rather than one figure, because the compiler can emit the routine at more than one level of detail, each a legitimate reading of $C_{\text{replay}}(k)$:

$$R = \frac{C_{\text{agent}}(k)}{C_{\text{replay}}(k)}, \quad C_{\text{replay}}(k) = \left|\texttt{tiktoken}(\cdot)\right|. \quad (3)$$

The compiler emits both rungs deterministically, with no model in the loop (`compile_replay.py`, released with the reference implementation [10]); tokens are counted with `tiktoken` under `cl100k_base`, the paper's encoding. The *operational* rung is the full *guarded skill plan*: a per-step sequence of typed actions, each carrying an expected-element, expected-role, and expected-application guard so replay can fail safe. Across the 20 most frequent action routines these plans have a median of 247.5 tokens and compile in a median of 0.5 ms at zero token cost, the compiler being CPU-only. The *ceiling* rung is the *minimal replay script* that strips the guards to the routine's bare information content: a median of 40 tokens for a median 5-step routine, about 8 tokens per step. The guarded plan is roughly six times larger, which is precisely why it yields the smaller, more defensible ratio we lead with.

7.2 The Ladder, and Its Distribution

We lead with the *conservative operational* number and keep the larger one as a ceiling, because leading with the ceiling would invite a cherry-picking objection. Against the guarded skill plan the compiler actually emits, the median is $R_{\text{inject}} = 60\times$ (interquartile 59–62 across the 20 compiled plans): injecting the compiled routine costs about one-sixtieth of re-deriving it from screenshots. Against the minimal script, the median reaches the information-content ceiling $R_{\text{info}} = 343\times$ at action granularity, the largest ratio the routine's bare description length can justify. Table 4 reports both rungs and, for R_{info}, the full distribution over routines. The ceiling is robust to the two knobs most likely to be accused of driving it: it rises monotonically but mildly with routine length (median 301× at k=3–4, 358× at k=5–8, 405× at k=9–15, 401× at $k\geq16$), and re-pricing the numerator at three screen configurations moves it only within a 259× (conservative 1280 × 800) to 425× (retina 1728 × 1117) band, so it is an artifact of neither short routines nor one display. At URL granularity the ceiling median is 198×. The most frequent action routine (a three-step compose-message loop, 181 occurrences) and the most frequent URL routine (a mail-calendar-mail triangle, 25 occurrences) are the mundane recurrences one would expect of real desk work, not exotic macros. We note the concentration plainly: the 20 action routines behind R_{inject} are dominated by two families, a compose-message loop and a window-close loop, sliced at several lengths by the n-gram miner, so R_{inject} characterizes this user's highest-frequency micro-routines rather than a diverse cross-application task set; Section 8.4 bounds the generalization.

Table 4: The denominator ladder for $R = C_{\text{agent}}/C_{\text{replay}}$ (numerator modeled; denominators measured with `cl100k_base`). R_{inject} (operational) uses the guarded skill plan the compiler emits; R_{info} (ceiling) uses the minimal replay script. R_{inject} is computed on the 20 most frequent action routines, so it carries no URL column. R_{info} medians are priced at a typical 1512 × 982 screen; the resolution band re-prices the numerator at a conservative and a retina capture.

	Action	URL
Recurring routines	614	261
Routine instances	5,508	1,303
R_{inject} **(guarded plan, operational)**	**60×**	n/a
IQR / median plan tokens	59–62× / 248	n/a
Compile cost B	0.5 ms, 0 tokens	
R_{info} (minimal script, ceiling)	343×	198×
Interquartile range	297–390×	165–228×
Recurrence-weighted mean	337×	194×
Min–max	167–509×	67–301×
Across screen resolutions	259–425×	150–245×
Median script tokens	40	43
Median steps k / max occ.	5 / 181	3 / 25

7.3 Recovery, and a Modeled Three-Arm Comparison

The recovery a hit yields depends on which rung of the ladder is used, and we attach the largest claim to the narrowest case. On a guard-matched step, deterministic *local* replay — *parametric routine replay*: the routine's compiled structure replays deterministically while its variable slots carry the request's new values, so the agent picks the routine and fills the slots but never re-derives the steps — takes the model out of the loop entirely, so it recovers

$$1 - \frac{1}{R} \approx 99\% \quad (4)$$

of that step's re-derivation cost (exactly 99.7% at the median ceiling $R_{\text{info}} = 343$). That 99% is a per-covered-step ceiling, not a fleet saving: local replay reaches it only on the fraction of steps the compiler can guard, whose measured median is 0.415, and it is exposed to interface drift that a live run must confirm (Section 8.4).

A modeled three-arm comparison makes the gap between ceiling and realized saving concrete. We price

three ways of executing the 20 recurring routines at Sonnet-class list rates ($3/$15 per Mtok, output fraction 0.07), *modeled from measured artifacts and token counts, not billed*: arm A, a memoryless agent that re-derives each routine; arm B, an agent given the compiled plan once and then acting with it in context; and arm C, deterministic local replay. Relative to arm A (32.8 Mtok, $125.81 over the fleet), injecting the plan (arm B) saves 83.3% of tokens ($20.96, a 6.0× per-occurrence reduction), and local replay (arm C) saves 40.8% ($74.46, 1.7×). Two honesty notes. Arm B's modeled saving assumes the screenshot baseline of Eq. 2; the one live in-loop comparison we have, against an accessibility-tree agent, saved only ≈14% (Section 8.4), so read 83.3% as a modeled upper bound, not a measured in-loop figure. Arm C falls well short of the 99% ceiling for a structural reason: it credits only the guard-matched fraction, so the complementary $1-0.415$ deopt fraction still pays full arm-A price. The live-billed version of this table, with real usage JSON, is reserved (Section 8.4); we present it as modeled.

Across *all* action steps rather than the recurring subset, the reachable saving is smaller still, because only a fraction h of steps sit in a delegable routine. The honest all-fleet ceiling is $h\,(1 - 1/R_{\text{info}})$: with the in-sample delegable rate $h = 9.0\%$ it is $\approx 9.0\%$, and with the conservative out-of-sample rate $h = 7.7\%$ (Section 7.4) it is $\approx 7.7\%$. Three disciplines keep every figure here honest. First, no rung of R is ever multiplied by the 86× context compression of Section 6: those are different tokens (reading a day versus re-deriving a routine), and stacking them would double-count the same work. Second, R is never combined with prompt-cache or KV-cache discounts; caching and replay are alternative recoveries of the same repetition, not additive ones. Third, both the numerator and the three-arm dollars are modeled, so we report them as ceilings and reserve the live billing.

7.4 Desktop Routine Recurrence h

The ratio R prices a single hit; the recurrence h is how often hits occur, measured as the fraction of action steps that fall inside a recurring routine. A single number would mislead, so we report h at two levels. The *raw* rate is $h_{\text{raw}} = 83.1\%$: the fraction of action steps inside *any* recurring n-gram. This is dominated by the generic micro-structure of input, the type-a-character, return-to-field, type-again texture that repeats constantly and carries no reusable work. The *delegable* rate is $h_{\text{specific}} = 9.0\%$ at action granularity (13.1% at URL granularity): the fraction of steps inside the $\geq$ 2-named-target routines of Section 7, that is, inside an identifiable, repeatable task. We report the gap $83.1\% \to 9.0\%$ openly rather than quoting the larger figure: it is the difference between "the keyboard repeats" and "the work repeats," and only the latter is a candidate for delegation.

A temporal holdout (the single predictive claim) A within-sample recurrence rate is partly circular, since a routine is counted as recurring in part because we already watched it recur. To obtain a non-circular value we fit the routine table on the first 40 active days (4,847 routine signatures) and then ask what fraction of action steps on the held-out final 11 days fall into a routine *already known* from the training window. The out-of-sample predicted hit rate is 7.7%, against an in-sample 8.6% on the training days themselves. The modest $8.6\% \to 7.7\%$ drop is the within-user temporal-drift gap (a same-user, later-in-time split, not a cross-population holdout), and 7.7% is the recurrence the cost accounting should carry; the all-fleet ceiling $h\,(1 - 1/R_{\text{info}}) \approx 7.7\%$ follows from it. We make exactly one predictive claim, and this is it.

What h is not Web-era studies of page revisitation report far larger constants, on the order of 40 to 58% of page visits being revisits [50, 51]. We do not claim those as h. A page revisit is not a delegable task: back-button noise, re-checking a feed, and reopening a tab are revisits with no reusable work content, and importing that number would overstate h by roughly an order of magnitude. Our h measures recurring *task* structure in real desktop work, and on this corpus that quantity is $\approx$ 0.08–0.13, not ≈ 0.5. Reporting the smaller, honest figure is the point of the specificity rule.

7.5 Reproducibility as a Certified Property

Because compilation contains no model, reproducibility is not a behavior we hope for but a property we can certify. A certification harness (`certify_ivm.py`, released with the code [10]) checks three conditions over the 51-active-day corpus and returns PASS. First, **byte-identical output**: re-compiling any day twice yields byte-identical documents once the single emission-metadata field `generated_at` (a wall-clock stamp of the run) is excluded. We name that exclusion rather than quietly canonicalizing it away; it is the one field that is not a function of the capture. Second, **rebuild equals incremental**: rebuilding the entire history from scratch produces the same bytes as compiling day-by-day, and earlier days are never rewritten by later capture (append-only). Third, **compile cost does not grow with history**: the median full-day compile across the run is 220.9 ms, and the mean over the second half of the corpus is 0.86× the mean over the first half (216.4 ms versus 251.8 ms), so per-day cost tracks the size of the day's delta, not the length of accumulated history; it is $O(|\Delta|)$. (The 68 ms of Section 6 is one representative lighter day; the certification spans light and heavy days, from 0.1 to 930 ms.)

We claim exactly one thing from this, and concede its lineage. The claim is a CI-checkable byte-equality con-

tract on a stateless projection of an append-only capture log. We claim *no* novelty in incremental view maintenance: deterministic, incrementally maintainable views over append-only logs are a mature area, with DBSP giving automatic incremental view maintenance for rich query languages [52], and event-sourcing long rebuilding state as a fold over an immutable event log. Our contribution is not the mechanism but its *use as a certified guarantee for agent memory*: an episodic-memory artifact whose equality across runs, across code versions, and across rebuild strategies is enforced by a test, which is what makes the memory safe to cache and mechanically auditable in the sense Section 8 requires.

8 Privacy, Trust, and Limitations

8.1 Privacy Model

The entire pipeline is local: capture, storage, and compilation happen on the user's machine, the compiler opens the capture database read-only, and nothing is transmitted anywhere by the system itself. The operator chooses what leaves, and when, by handing a compiled artifact to an agent. The schema-level content rule (Section 3) keeps typed text out of documents by default; audio capture is off by default in the provisioned engine. Compilation itself involves no model of any kind; the capture engine does run on-device OCR to read screen content, but that output never leaves the machine, and no language model, local or remote, participates in producing memory. The capture database itself remains sensitive at rest, as membership-inference work on memory stores reminds us [36]; we treat device-level encryption as the appropriate control and note that this risk is shared by every capture system rather than introduced by compilation.

8.2 Trust Properties

Two structural properties address emerging attacks on agent memory. Evidence pointers make every episode mechanically auditable: a verifier can re-read the named raw rows and recompute the frame, which raises the bar for poisoning attacks that rely on unverifiable memories [35]. The measured/inferred boundary ensures that even a compromised or careless tier-2 tool cannot inject interpretation disguised as observation; consumers can always strip to the measured core. We do not claim these properties defeat a compromised capture engine, which can fabricate raw rows; provenance begins at the database.

8.3 Limitations

Four limitations bound our claims. First, the empirical characterization is a single-user corpus (one professional, one machine, 61 days); cadence, fragmentation, and entity coverage will differ across roles and platforms, and a multi-user study is future work. Second, the reference implementation reads one capture engine's database layout; the schema is engine-agnostic but each new source needs an adapter. Third, the measured tier is structurally silent on intent: it reports two profile views and a people search, never "prospecting," and consumers who need intent must add tier-2 inference and accept its tags. Fourth, screen presence is an imperfect proxy for attention, and on this corpus the gap is quantifiable rather than hypothetical. The capture heartbeat keeps input-free stretches credited: intervals that terminate in a heartbeat row carry 45% of all credited active time, and uninterrupted heartbeat runs reach 42 minutes, so dwell includes reading and watching but also time the user had stepped away from an awake display. The dwell cap does not bound this; it only bounds credit across capture stalls and display sleep. Frames report input volume precisely so consumers can gate on interaction, and a tier-2 tool may label presence-only stretches, but the measured tier reports tenure, not attention, and consumers must read it that way. Monitors compound the proxy error in the other direction: each records its own stream, so a minute with two active monitors earns credit twice in per-application ledgers (27% of captured minutes here; Section 6.2). Off-screen work (paper, phone, conversation) appears only as gaps. Relatedly, frames measure attention episodes, not tasks: interruption research shows interrupted tasks are resumed across much longer horizons than any single frame [23], so task-level structure belongs to tier-2 inference. Finally, the downstream benchmark (Section 6.2) evaluates two agent tiers (Claude Sonnet 4.5 and Opus 4.5) on this single-user corpus with one answering pass each. A stronger model narrows the summary's gap, so the magnitude of the effect is not model-independent; but the block's determinism, its context-fit, and its being read equally well by both tiers are structural properties no model strength supplies, and we release the harness so the comparison can be rerun with confidence intervals, against other model families such as GPT and Gemini, and, as multi-user capture becomes available, other users.

8.4 Limitations of the Overhead Measurements

Five limitations bound the results of this half of the paper, beyond the systems limitations already stated in Section 8.

Single user. Every number in Section 7 comes from one user's machine, the author's, over 51 active days (128,756 frames). We state this plainly: R and h are properties of this person's work, not of a population. The specificity rule and the temporal holdout guard against granularity and circularity artifacts, but

not against having sampled one professional on one platform. A multi-user replication, and a public-dataset second subject, are the obvious next step and are not claimed here.

The numerator and the three-arm dollars are modeled, not billed. R's numerator (Eq. 2) is what a memoryless screenshot-driven loop would spend under the Anthropic image-token rule and fixed per-step budgets, priced without running an agent, and the three-arm comparison of Section 7 is likewise modeled from measured artifacts (compiled plans, token counts, guard coverage) at list prices, not billed. The denominators themselves are real: the guarded plan and the minimal script are emitted and tokenized deterministically, and the median guard coverage of 0.415 that bounds local replay (arm C) is measured, not assumed. Since the freeze, one owner-authorized live execution has been run, and we report it as a first confirmation of the replay side. A compiled two-step routine (open a compose surface, type a draft; nothing was ever posted) was matched to a natural-language request by a small *local* model (a Tier-2 retrieval step we do not count in the execution total) and then executed in a live, authenticated browser session by the released executor: both steps grounded by accessibility role+name, *zero* model tokens *at execution*, a few seconds of wall-clock. The element references differed between two runs of the same plan and name-based grounding adapted; on a wrong page the same plan grounded nothing and performed zero actions, the intended fail-safe. The accessibility snapshots an in-loop agent would read to choose each action measured ≈10.5k tokens per step on the same pages; a separate live comparison that kept the model *in* the loop with the compiled plan as context saved only ≈14% against an accessibility-driven agent, so the large ratios of Section 7 require the model fully out of the loop, which parametric replay is. Three bounds on this confirmation: it is one two-step routine; its plan was seeded from live accessibility names standing in for a mined routine (click-level grounding of the recorder is still under validation); and the guard-miss deopt path was not exercised. The full live three-arm billing with real usage JSON remains reserved. Read R as a modeled ratio whose denominator — including its zero-token on-hit case — is now live-confirmed, and the dollar savings as a modeled ceiling awaiting live billing.

Capture is not free. The instrument has an operating cost, which we report rather than hide. The corpus database is 9.5 GB, about 0.19 GB per active day; on-device OCR runs on 96% of frames, a continuous duty cycle; and the accessibility tree that grounds replay is present on 81.5% of frames, so 18.5% offer no structured target and would fall back to coordinates. Replay coverage is therefore bounded by that 81.5%, and the compression and QA wins of Section 6 are gross of the capture engine's own footprint.

Match precision propagates into q. Turning a captured routine into a replayable script depends on entity typing, which is approximately 82% accurate at assigning a non-generic type (Section 6). Typing errors propagate directly into the match-precision term q of Eq. 1: a mistyped target can match the wrong routine or fail to match a real one, so $q < 1$ and the honest fleet saving carries that factor. We do not assume $q = 1$.

OCR is a model, so determinism holds forward, not back to pixels. The only learned component anywhere in the pipeline is the on-device OCR that reads screen text at capture time. Byte-level reproducibility (Section 7.5) therefore holds from the *stored OCR text* forward: given the captured text, every downstream document is a deterministic function of it. It does not hold back to the pixels, because a different OCR model, or a re-scan of the same screenshots, could yield different text. We scope the determinism claim to the compile path over stored capture, never to perception.

9 Conclusion

Agents are blind to the activity stream that most defines their user's day, not because capture is missing but because nothing turns capture into memory an agent can afford, reproduce, and trust. Activity frames fill that seam with the least interesting tool available, deterministic code, and we argue that this dullness is the point: at 68 ms and zero tokens per day, episodic memory becomes infrastructure rather than inference, and the measured/inferred boundary gives interpretation a place to live without contaminating fact. The same deterministic compiler doubles as a demand-side instrument: because it reduces recurring capture to a replayable script by code alone, it reads the cost parameters R and h that agent-cost models assume, here as first single-user values, with a modeled numerator, awaiting the multi-user replication and live billing we reserve. The schema, compilation rules, and implementation are open [10]; we hope the format outlives the reference code, and that capture systems, memory layers, and agents converge on a shared, honest representation of what a person actually did.

References

[1] M. Pink, Q. Wu, V. A. Vo, J. Turek, J. Mu, A. Huth, and M. Toneva, "Position: Episodic memory is the missing piece for long-term LLM agents," 2025.

[2] C. Zhou, H. Chai, W. Chen, Z. Guo, R. Shan, Y. Song, T. Xu, Y. Yang, A. Yu, W. Zhang, C. Zheng, J. Zhu, Z. Zheng, Z. Zhang, X. Lou, C. Zhang, Z. Fu, J. Wang, W. Liu, J. Lin, and W. Zhang, "Externalization in LLM agents: A unified review of memory, skills, protocols and harness engineering," 2026.

[3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2024.

[4] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," 2025.

[5] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," 2025.

[6] E. Bjäreholt and J. Nilsson, "ActivityWatch: open-source automated time tracker." `https://activitywatch.net`, 2016. Accessed 2026-07-04.

[7] Microsoft, "Recall: retrace your steps on Copilot+ PCs." `https://learn.microsoft.com/en-us/windows/apps/develop/windows-integration/recall/`, 2025. Accessed 2026-07-04.

[8] OpenAI, "Chronicle: memories from recent screen content in Codex for macOS." `https://developers.openai.com/codex/memories/chronicle`, 2026. Research preview, accessed 2026-07-04.

[9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," 2023.

[10] N. Iyamu, "activity-frames: episodic memory for AI agents." `https://github.com/nossa-y/activity-frames`, 2026. MIT license, accessed 2026-07-04.

[11] Anthropic, "Model Context Protocol." `https://modelcontextprotocol.io`, 2024. Open protocol specification, accessed 2026-07-04.

[12] K. Wang, Y. Lin, J. Lou, Z. Zhou, B. Suvonov, and J. Li, "E-mem: Multi-agent based episodic context reconstruction for LLM agent memory," 2026.

[13] S. Forouzandeh, W. Peng, P. Moradi, X. Yu, and M. Jalili, "Learning hierarchical procedural memory for LLM agents through bayesian selection and contrastive refinement," 2025.

[14] Z. Pan, Q. Wu, H. Jiang, X. Luo, H. Cheng, D. Li, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and J. Gao, "On memory construction and retrieval for personalized conversational agents," 2025.

[15] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent workflow memory," 2024.

[16] Y. Wang and X. Chen, "Mirix: Multi-agent memory system for llm-based agents," 2025.

[17] H. Yin, Z. Wen, J. Cao, B. Yuan, and R. Yang, "Focal: Filtered on-device continuous activity logging for efficient personal desktop summarization," 2026.

[18] Y. Tang, H. Tang, T. Cao, L. Nguyen, A. Zhang, X. Cao, C. Liu, W. Ding, and Y. Li, "Proagentbench: Evaluating llm agents for proactive assistance with real-world data," 2026.

[19] G. Zhang, M. Ahmed, Z. Hu, and A. Bulling, "Summact: Uncovering user intentions through interactive behaviour summarisation," 2024.

[20] J. N. Li, Z. J. Zhang, and J. Ma, "Omniquery: Contextually augmenting captured multimodal memory to enable personal question answering," 2025.

[21] A. N. Dragunov, T. G. Dietterich, K. Johnsrude, M. McLaughlin, L. Li, and J. L. Herlocker, "TaskTracer: A desktop environment to support multitasking knowledge workers," in *Proc. IUI*, pp. 75–82, 2005.

[22] N. Oliver, G. Smith, C. Thakkar, and A. C. Surendran, "SWISH: Semantic analysis of window titles and switching history," in *Proc. IUI*, pp. 92–99, 2006.

[23] V. M. González and G. Mark, ""constant, constant, multi-tasking craziness": Managing multiple working spheres," in *Proc. CHI*, pp. 113–120, 2004.

[24] L. Yeykelis, J. J. Cummings, and B. Reeves, "Multitasking on a single device: Arousal and the frequency, anticipation, and prediction of switching between media content on a computer," *Journal of Communication*, vol. 64, no. 1, pp. 167–192, 2014.

[25] A. R. Doherty and A. F. Smeaton, "Automatically segmenting LifeLog data into events," in *Proc. WIAMIS*, pp. 20–23, 2008.

[26] A. J. Sellen and S. Whittaker, "Beyond total capture: A constructive critique of lifelogging," *Communications of the ACM*, vol. 53, no. 5, pp. 70–77, 2010.

[27] R. Cooley, B. Mobasher, and J. Srivastava, "Data preparation for mining world wide web browsing patterns," *Knowledge and Information Systems*, vol. 1, no. 1, pp. 5–32, 1999.

[28] S. J. van Zelst, F. Mannhardt, M. de Leoni, and A. Koschmider, "Event abstraction in process mining: Literature review and taxonomy," *Granular Computing*, vol. 6, pp. 719–736, 2021.

[29] V. Leno, A. Polyvyanyy, M. Dumas, M. La Rosa, and F. M. Maggi, "Robotic process mining: Vision and challenges," *Business & Information Systems Engineering*, vol. 63, no. 3, pp. 301–314, 2021.

[30] F. Portet, E. Reiter, A. Gatt, J. Hunter, S. Sripada, Y. Freer, and C. Sykes, "Automatic generation of textual summaries from neonatal intensive care data," *Artificial Intelligence*, vol. 173, no. 7–8, pp. 789–816, 2009.

[31] C. Zhang, S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, G. Liu, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, "Large language model-brained GUI agents: A survey," 2025.

[32] G. Liu, P. Zhao, Y. Liang, L. Liu, Y. Guo, H. Xiao, W. Lin, Y. Chai, Y. Han, S. Ren, H. Wang, X. Liang, W. Wang, T. Wu, Z. Lu, S. Chen, LiLinghao, H. Wang, G. Xiong, Y. Liu, and H. Li, "LLM-powered GUI agents in phone automation: Surveying progress and prospects," 2025.

[33] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," 2023.

[34] Y. Li, H. Wen, W. Wang, X. Li, Y. Yuan, G. Liu, J. Liu, W. Xu, X. Wang, Y. Sun, R. Kong, Y. Wang, H. Geng, J. Luan, X. Jin, Z. Ye, G. Xiong, F. Zhang, X. Li, M. Xu, Z. Li, P. Li, Y. Liu, Y.-Q. Zhang, and Y. Liu, "Personal LLM agents: Insights and survey about the capability, efficiency and security," 2024.

[35] Y. Louck, "Securing LLM-agent long-term memory against poisoning: Non-malleable, origin-bound authority with machine-checked guarantees," 2026.

[36] K. Chen, Y. Pang, and T. Wang, "MRMMIA: Membership inference attacks on memory in chat agents," 2026.

[37] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," 2025.

[38] B. Zheng, M. Y. Fatemi, X. Jin, *et al.*, "SkillWeaver: Web agents can self-improve by discovering and honing skills," 2025.

[39] Z. Z. Wang, A. Gandhi, G. Neubig, *et al.*, "Inducing programmatic skills for agentic tasks," 2025.

[40] B. Li, "PreAct: Computer-using agents that get faster on repeated tasks," 2026.

[41] V. Pahuja, Y. Lu, C. Rosset, B. Gou, A. Mitra, S. Whitehead, Y. Su, and A. Awadallah, "Explorer: Scaling exploration-driven web trajectory synthesis for multimodal web agents," 2025.

[42] Y. Xu, D. Lu, Z. Shen, J. Wang, Z. Wang, Y. Mao, C. Xiong, and T. Yu, "AgentTrek: Agent trajectory synthesis via guiding replay with web tutorials," 2024.

[43] C. H. Song, Y. Song, P. Goyal, Y. Su, O. Riva, H. Palangi, and T. Pfister, "Watch and learn: Learning to use computers from online videos," 2025.

[44] M. Oyamada, K. Takeoka, K. Akimoto, R. Obara, M. Enomoto, H. Zhang, D. Haraguchi, and T. Tamura, "cotomi act: Learning to automate work by watching you," 2026.

[45] Y. Song, K. Ramaneti, Z. Sheikh, *et al.*, "Agent data protocol: Unifying datasets for diverse, effective fine-tuning of LLM agents," 2025.

[46] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," 2023.

[47] M. H. Erol, B. El, M. Suzgun, M. Yuksekgonul, and J. Zou, "Cost-of-pass: An economic framework for evaluating language models," 2025.

[48] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI agents that matter," 2024.

[49] S. Kapoor, B. Stroebl, P. Kirgis, N. Nadgir, Z. S. Siegel, B. Wei, T. Xue, Z. Chen, F. Chen, S. Utpala, F. Ndzomga, D. Oruganty, S. Luskin, K. Liu, B. Yu, A. Arora, D. Hahm, H. Trivedi, H. Sun, J. Lee, T. Jin, Y. Mai, Y. Zhou, Y. Zhu, R. Bommasani, D. Kang, D. Song, P. Henderson, Y. Su, P. Liang, and A. Narayanan, "Holistic agent leaderboard: The missing infrastructure for AI agent evaluation," 2025.

[50] L. Tauscher and S. Greenberg, "How people revisit web pages: empirical findings and implications for the design of history systems," *International Journal of Human-Computer Studies*, vol. 47, no. 1, pp. 97–137, 1997.

[51] E. Adar, J. Teevan, and S. T. Dumais, "Large scale analysis of web revisitation patterns," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pp. 1197–1206, 2008.

[52] M. Budiu, F. McSherry, L. Ryzhyk, and V. Tannen, "DBSP: Automatic incremental view maintenance for rich query languages," 2022.